

Section 77 — Suffix Automata And Palindromic Trees

1. Two compressed substring graphs

A suffix automaton (SAM) merges substrings having the same set of ending positions. Its transitions form a DAG because transition lengths strictly increase. A palindromic tree keeps one node per distinct palindrome and links it to its longest proper palindromic suffix.

This lesson develops Problems A–C. Problem D is intentionally explained only in the editorial.

SAM vocabulary and clones

For state v , $\text{len}[v]$ is the maximum represented length and $\text{link}[v]$ is the class of its longest proper suffix. The state represents every length from $\text{len}[\text{link}[v]]+1$ through $\text{len}[v]$. A transition appends a character; a suffix link removes leading characters to reach a less-specific end-position class.

When extension would violate the required length relation, a **clone** copies transitions and a suffix link but starts with zero direct occurrences. Redirecting transitions to it splits a former class by right context. Clones are what preserve both determinism and the linear state bound.

2. Problem A: k-th distinct substring

Every nonempty path from the SAM root spells one distinct substring. Let $\text{dp}[v]$ be the number of nonempty paths starting at state v :

$$\text{dp}[v] = \text{sum over transitions } v \rightarrow u \text{ of } (1 + \text{dp}[u]).$$

Process states by decreasing maximum length. For reconstruction, inspect outgoing letters in sorted order. A transition owns one substring ending immediately there, followed by all extensions below it. Subtract whole blocks until the block containing k is found.

The state is a compressed model: exponentially many substring occurrences collapse to at most $2n-1$ end-position classes.

If the root's a transition reaches a state with continuations b and c , that transition owns substring a plus every extension below it. Its block size is $1+\text{dp}[\text{next}]$. After selecting the transition, stop when $k=1$; otherwise decrement once and continue. Cap counts above the largest queried k to avoid overflow in weighted variants.

3. Problem B: longest common substring

Build the SAM of **a**, then scan **b**. Maintain a state and the length of the longest suffix of the scanned prefix that occurs in **a**. On a missing transition, follow suffix links until the character fits, shortening the current match to the new state's maximum length.

Invariant. After each scanned character, **length** is the longest suffix of the processed prefix of **b** represented by **state**.

Each suffix-link retreat can be amortized against progress, so the scan is linear.

While scanning a second string, a missing transition does not restart from the next position. Suffix links try shorter suffixes ending at the same current position. Once an edge fits, first limit the matched length to the new state's **len**, then append the character. Every retreat removes already matched characters, which supplies the amortized linear bound.

4. Problem C: palindrome prefix profile

When one character is appended, at most one new distinct palindrome appears: every new palindrome must end at the new position, and only the longest new palindromic suffix can be new.

To add a character:

1. follow suffix links from the previous longest palindromic suffix until the new character also matches on its left;
2. reuse its transition if present;
3. otherwise create one node and compute its suffix link similarly.

The two roots of lengths -1 and 0 remove parity special cases. The current node directly gives the longest palindromic suffix; nodes minus two gives the distinct count.

The imaginary length -1 root always permits a new one-character palindrome: its left comparison lands on the character just appended. The length-zero root handles even palindromes. With both roots, the same suffix-link loop handles the first character, odd palindromes, and even palindromes.

Distinct objects versus occurrences

One SAM path or one palindromic-tree node represents a distinct string, not one occurrence. To count occurrences, mark the state reached after every appended prefix and propagate counts from longer states toward suffix links. Longer-first order is essential because every occurrence is also an occurrence of each represented suffix. Clone occurrence count starts at zero.

Implementation notes

Arrays of 26 transitions are predictable for lowercase input; dictionaries save memory for larger alphabets. Python generally benefits from dictionaries per state. Keep states in creation order, then sort indices by **len** for link propagation. For an eertree, store a sentinel before the text so the left-side comparison follows one uniform rule.

Proof habits and stress tests

- Define the complete set of strings represented by a state.
- Prove the bijection between root paths and distinct substrings.
- State the longest-suffix invariant after each streamed character.
- Prove why longer-to-shorter link propagation counts every suffix occurrence.

For tiny strings, enumerate all substrings and palindromes literally. Compare lexicographic lists, occurrence counts, common substrings, and absent words. Include unary strings, alternating strings, clone-heavy strings such as `abbb`, and long palindromic borders.

5. Choosing the structure

- Need lexicographic enumeration, suffix intervals, or LCP geometry: suffix array.
- Need online extension, occurrence classes, or path DP over all substrings: suffix automaton.
- Need one node per distinct palindrome and palindromic suffix links: palindromic tree.

Exercises

- A — path counting and lexicographic reconstruction.
- B — streaming matches through suffix links.
- C — online palindromic suffix maintenance.
- D — a fresh automaton-graph problem reserved for the editorial.